

Week 4

從 Autoencoder 到 U-Net

壓縮 → 還原 → 補回細節：三個心智模型

MLP autoencoder

CNN autoencoder + ConvT2d

U-Net + skip connection

這週要回答的問題

到目前為止，模型的輸出都是一個標籤。如果輸出也必須是一張影像呢？

Week 3 — 分類

28×28 影像 $\rightarrow$ 10 個數字

- 整條網路都在「丟資訊」
- 只要保留能分辨類別的部分
- 只需要 encoder

Week 4 — 重建

28×28 影像 $\rightarrow$ 28×28 影像

- 壓縮之後還要「長回來」
- 每一個像素都要交代
- 需要一個 decoder — 這是新的一半

分類只需要學會「壓縮」。重建要同時學會「壓縮」和「還原」——這週補的就是還原這一半。

三個心智模型

每一步只加入一個新概念，其餘全部沿用 Week 1 的訓練迴圈

1

MLP autoencoder

「瓶頸」

強迫資訊擠過一個很窄的通道，模型只好學會抓重點。

新概念：bottleneck

2

CNN autoencoder

「散射」

卷積是多對一地讀，轉置卷積是一對多地寫回去。

新概念：upsampling

3

U-Net

「旁路」

細節在下採樣時就丟了；開一條路把它直接送到 decoder。

新概念：skip connection

三個模型的訓練迴圈完全一樣，差別只在 forward 裡怎麼把資訊送回去。

PART 1

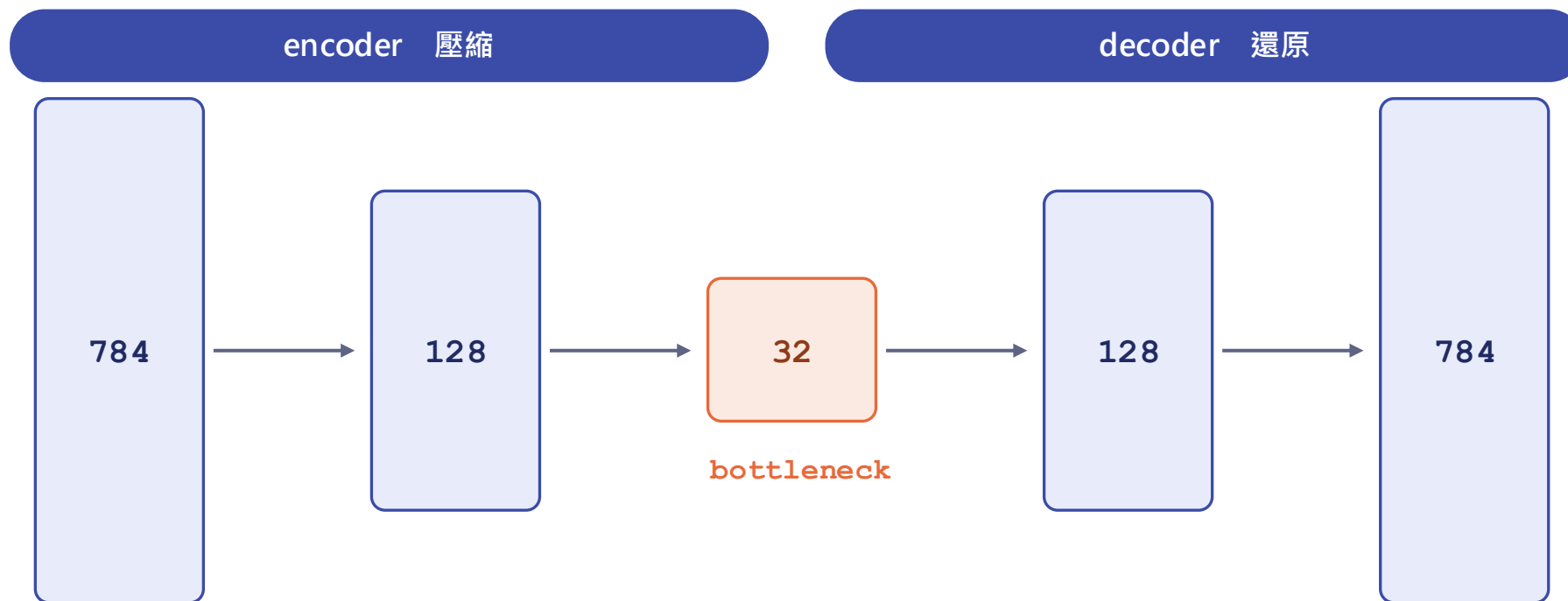
MLP autoencoder

心智模型：瓶頸——只有摘要能通過的通道

01

架構：一個對稱的漏斗

input = target = 同一張圖，不需要任何標籤



整個 784 維的資訊必須先擠過 32 個數字，再重新展開。能還原，代表那 32 個數字裡確實裝下了這張圖的重點。

心智模型：只有摘要能通過

以及一個讓學生驚訝的事實 —— 訓練迴圈幾乎沒改

把一本書濃縮成 3 句話，再請人只憑那 3 句話把書重寫出來。

- 寫得回來 → 那 3 句話抓對了重點
- 瓶頸太寬 → 模型直接抄，什麼也沒學到
- 瓶頸太窄 → 重點也裝不下，重建糊掉
- 監督訊號來自資料本身，不需要標籤

跟 Week 1 Part 3 的差別

```
for x, _ in loader:
    recon = model(x)
    loss = loss_fn(recon, x)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

唯一改的一行：target 從 y 變成 x 自己。

最後一層接 Sigmoid，因為像素值在 [0, 1]。
loss 用 MSELoss。

實驗：把瓶頸愈掐愈緊

掃過不同的 bottleneck 維度，並排看重建結果

128

維 bottleneck

幾乎完美

但沒學到東西 —— 接近直接複製

32

維 bottleneck

清楚可辨

壓縮 24 倍，細節仍在

8

維 bottleneck

認得出數字

筆畫粗細與風格開始流失

2

維 bottleneck

只剩大致形狀

但可以把 latent 畫成一張二維地圖

回頭呼應 Week 2 Part 4

把所有 ReLU 拿掉，這個 autoencoder 學到的東西就等價於 PCA。加回非線性之後，它能做到 PCA 做不到的事 —— 這就是「學出來的降維」。

PART 2

CNN autoencoder + ConvTranspose2d

心智模型：卷積是 gather，轉置卷積是 scatter

MLP autoencoder 的第一行就出賣了它

`x.view(-1, 784)` —— 二維結構在這裡被壓平丟掉了

攤平之後

- 相鄰的兩個像素，跟隔了一整列的像素，對模型來說一樣遠
- 每個位置都要學自己的一組權重
- $784 \times 128 \approx 10$ 萬個參數，只是第一層

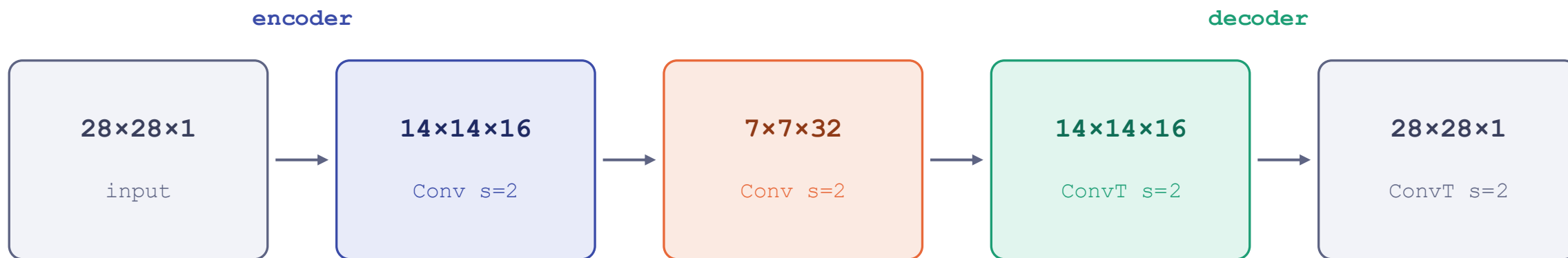
換成卷積之後

- 空間結構被保留，鄰近像素依然鄰近
- 同一組 kernel 在整張圖上共用
- 參數少一個數量級，重建反而更好

Week 3 已經教過怎麼用卷積「讀」影像。問題只剩一個：decoder 要怎麼把 7×7 變回 28×28 ？

CNN autoencoder：一樣的漏斗，換成卷積

下採樣用 stride=2，上採樣用 ConvTranspose2d



已知：怎麼變小

stride=2 的卷積，或 MaxPool。Week 3 教過了。

新的：怎麼變大

ConvTranspose2d —— 這是整堂課唯一需要慢慢講的新 layer。

ConvTranspose2d 的心智模型

不要想成「反卷積」。想成方向反過來的同一件事。

Conv2d

gather

讀一整塊 $k \times k$ 區域

寫出一個數字

多對一 → 尺寸變小

ConvTranspose2d

scatter

讀一個數字

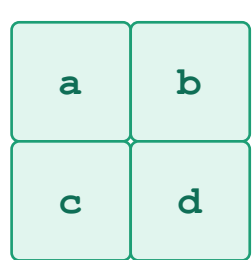
寫出一整塊 $k \times k$ ，重疊的地方相加

一對多 → 尺寸變大

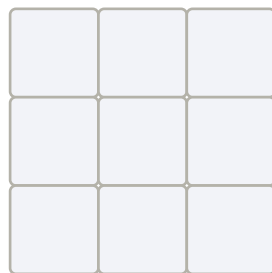
stride 在這裡不是「掃描步長」，而是「兩個蓋章之間的間距」。

機制：蓋章，然後相加

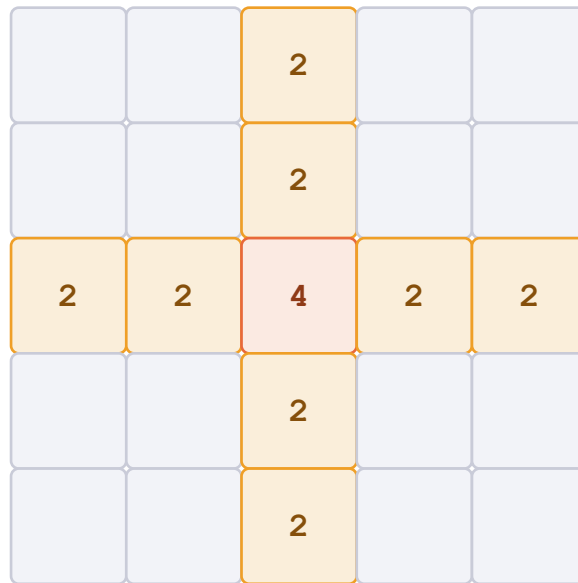
輸入 2×2 · kernel 3×3 · stride 2 → 輸出 5×5



input 2×2



kernel 3×3



output 5×5 (格內數字 = 疊加次數)

每個輸入像素乘上 kernel、蓋在輸出上，蓋章間距 = stride，重疊處相加。

重疊次數不均勻 (1 / 2 / 4) —— 這正是下一張「棋盤格 artifact」的來源。

順手考一題：如果 stride 改成 1，輸出會是幾乘幾？

輸出尺寸怎麼算

跟 Conv2d 的公式互為反函數

$$\text{out} = (\text{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{kernel_size} + \text{output_padding}$$

設定	代入	結果
k=2, s=2, p=0	$(7-1) \times 2 + 2$	7 → 14 ✓
k=3, s=2, p=1, op=1	$(7-1) \times 2 - 2 + 3 + 1$	7 → 14 ✓
k=4, s=2, p=1	$(7-1) \times 2 - 2 + 4$	7 → 14 ✓

教材裡 decoder 的 $7 \rightarrow 14 \rightarrow 28$ ，用上表任一組設定都可以。建議先用 $k=2, s=2$ 。

小提醒

padding 在這裡是「裁掉」，不是「補上」

1

padding=p 會從輸出的四邊各削掉 p 行 / p 列。因為它對應的是 forward conv 曾經補過的那圈 zero padding，反向操作自然是把它移除。

output_padding 存在，是因為尺寸映射不可逆

2

stride=2 的卷積會把 7×7 和 8×8 map 到同一個輸出尺寸（公式裡有 floor）。反過來時 PyTorch 無從判斷你要 14 還是 15，output_padding 就是你來告訴它。它只補在下緣和右緣，且不參與運算。

它叫 transposed，不叫 deconvolution

3

卷積可以寫成矩陣乘法 $y = Cx$ 。轉置卷積算的是 $x' = C^T y$ —— 連接關係一模一樣，只是方向反過來。它還原的是「形狀」，不是原本的「數值」。

棋盤格 artifact 從哪裡來

上一張蓋章圖已經回答了：疊加次數不均勻

kernel_size 不能被 stride 整除時

輸出各位置收到的貢獻次數不同 (1 次 / 2 次 / 4 次)
，梯度也跟著不均勻，訓練後就在影像上留下規律的方格紋。

三種修法

- 改成 $k=2, s=2$ 或 $k=4, s=2$ (整除)
- Upsample(nearest) + Conv2d($k=3, p=1$)
- Upsample(bilinear) + Conv2d · 更平滑但略糊

課堂建議

先用 $k=3, s=2, p=1, op=1$ 故意跑出棋盤格，讓學生親眼看到 artifact，再換成 Upsample + Conv 對照。

「先弄壞，再修好」比一開始就給正確答案有效——而且跟後面 U-Net 的「先模糊，再加 skip」是同一個節奏。

PART 3

U-Net

心智模型：旁路——細節不必擠過瓶頸

先讓 CNN autoencoder 失敗一次

任務換成去噪：input = 加了雜訊的圖，target = 乾淨原圖



為什麼會糊？

所有資訊都被迫擠過 7×7 的 bottleneck。高頻的東西 —— 邊緣、筆畫粗細 —— 在下採樣時就已經丟了，decoder 沒辦法無中生有。

那.....

encoder 早期的 feature map 明明還留著這些細節，為什麼不直接拿過來用？

U-Net 的心智模型：開一條旁路

瓶頸負責「這是什麼」，旁路負責「它長在哪裡」

搬家時，貴重物品不塞進壓縮袋——另外拿一個袋子直接提過去。

走瓶頸的路

`encoder → bottleneck → decoder`

帶的是「這張圖是什麼」——語意、整體結構。經過多次下採樣，視野大但位置模糊。

走旁路的路

`encoder → concat → decoder`

帶的是「東西在哪裡」——邊緣、紋理、精確位置。完全沒被壓縮過，解析度原封不動。

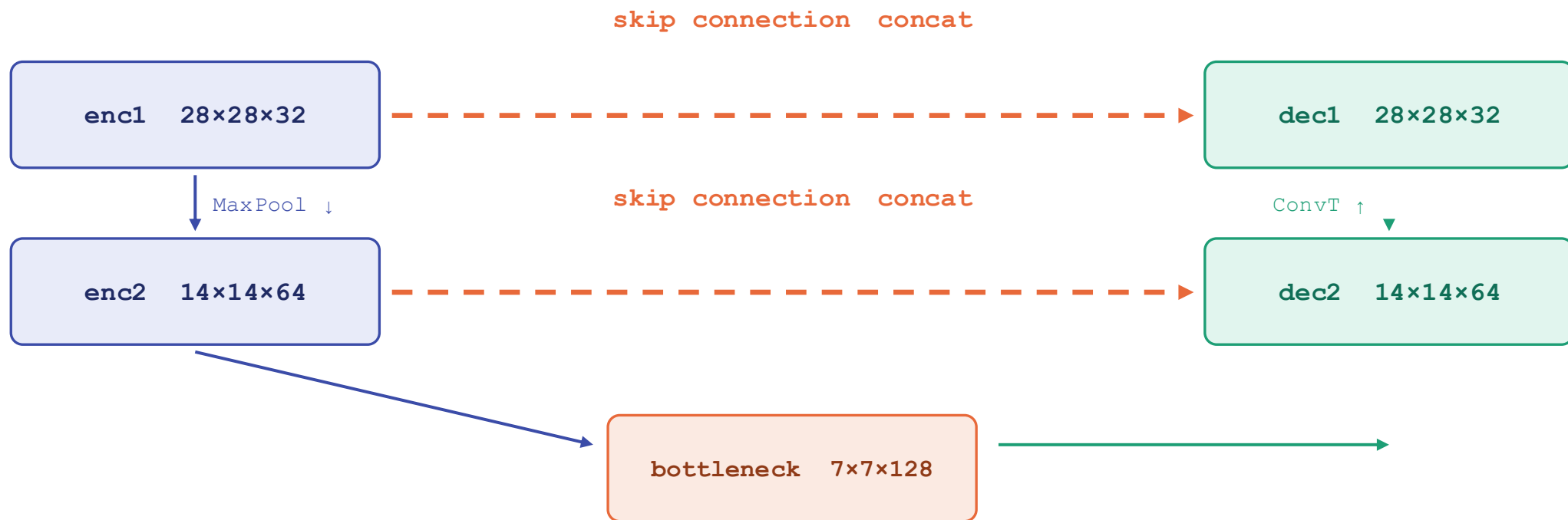
decoder 的工作

把兩者合起來

拿語意決定「要畫什麼」，拿旁路決定「畫在哪個像素」。這就是 U-Net 比 CNN AE 銳利的原因。

架構：為什麼長成一個 U 字

左右對稱不是裝飾 —— 是為了讓每一層 skip 的解析度剛好對得上



每個 block 都是 DoubleConv (Conv3×3 → BN → ReLU , 做兩次) 。decoder 的輸入通道會加倍 (64 → 128) , 因為 concat 之後通道數是兩份相加 —— 這是學生最常寫錯的一行。

concat，不是 add

以及怎麼證明效果真的來自 skip connection

U-Net : concat

```
torch.cat([up, skip], dim=1)
```

沿通道維度串接。兩份資訊並排放著，讓後面的卷積自己決定要用多少。通道數相加。

ResNet : add

```
out = f(x) + x
```

逐元素相加。兩份資訊被混在一起，無法再分開。通道數不變。

消融實驗（一定要做）

把 `torch.cat` 那幾行註解掉、decoder 通道數改回一半，其他完全不動，重跑一次。

如果不做這一步，學生會以為 U-Net 只是「比較大的 CNN autoencoder」，效果來自參數變多。

這週的三句話

1

`bottleneck`

強迫資訊擠過窄通道，模型只好學會抓重點。

2

`ConvTranspose2d`

卷積是 `gather`，轉置卷積是 `scatter` —— 蓋章，然後相加。

3

`skip connection`

細節不必擠過瓶頸，開一條旁路直接送到 `decoder`。

下一步：把同一套 U-Net 換到 `inpainting`、`super-resolution` 或影像分割 —— 架構一行都不用改。